

Relatório Técnico de Desenvolvimento: Módulo AVD1 – ASP.NET Core MVC

Identificação Acadêmica

- Discente: Ocimar Pereira de Souza Junior
- Matrícula: 202311030
- Curso: Sistemas de Informação

1. Introdução

O presente documento técnico tem como objetivo relatar o processo de desenvolvimento e a resolução das atividades práticas e teóricas propostas durante o módulo AVD1. O escopo principal deste módulo compreende a introdução aos conceitos fundamentais e a implementação de um sistema CRUD (Create, Read, Update, Delete) empregando o framework ASP.NET Core MVC.

A estrutura deste relatório foi delineada de modo a refletir a progressão didática dos materiais fornecidos, detalhando as abordagens adotadas para a resolução das questões teóricas e a arquitetura das soluções práticas. Todo o código-fonte produzido ao longo destas atividades encontra-se devidamente versionado e pode ser acessado por meio do repositório institucional no GitHub.

2. Metodologia e Desenvolvimento das Atividades

Nesta seção, são expostas as metodologias aplicadas na resolução dos exercícios, organizadas conforme os módulos didáticos disponibilizados.

2.1. Fundamentos do ASP.NET Core (Arquivo 1)

Atividade Prática (Slide 36): Estruturação Visual do Registro de Alunos

A etapa inicial consistiu na modelagem da entidade `Aluno`, contemplando os atributos essenciais: Nome, Email, Telefone, Endereço, Complemento, Bairro, Município, UF e CEP. Posteriormente, procedeu-se com a implementação do `AcademicoController` e da respectiva interface de usuário (`Aluno.cshtml`). A interface foi construída utilizando os recursos de estilização do framework Bootstrap (classes como `form-group`, `form-control` e `panel`), visando garantir a fidelidade visual em relação ao protótipo apresentado no material didático.

2.2. Implementação de CRUD Básico (Arquivo 2)

Atividades Práticas (Slides 9 a 24): Views, Controllers e Padrão DAO

O objetivo desta fase foi estabelecer as operações fundamentais de cadastro e listagem, integrando Tag Helpers e o padrão de projeto Data Access Object (DAO).

- Interface de Cadastro: O formulário foi refatorado para adotar a diretiva `@model Aluno`, empregando Tag Helpers (`asp-for`) para otimizar a vinculação de dados.
- Interface de Listagem (Index): Desenvolveu-se a View `Index.cshtml`, responsável por renderizar os registros em formato tabular HTML, utilizando a tipagem `@model IEnumerable<Academico.Models.Aluno>`.
- Controlador (AlunoController): O controlador foi expandido com os métodos `Create` (suportando requisições GET e POST) e `Index`. Foram aplicadas anotações de segurança e vinculação, tais como `[HttpPost]`, `[ValidateAntiForgeryToken]` e `[Bind]`.
- Persistência de Dados (Padrão DAO): Para o encapsulamento do acesso aos dados, implementou-se a classe `AlunoDao`. Nesta etapa, a persistência foi simulada em memória através de uma lista estática, provendo os métodos `GravarAluno` e `ObterTodos`.

2.3. Validações e Conceitos Teóricos (Arquivo 3)

Atividades Teóricas (Slide 27): Aplicação de Data Annotations

A validação de dados foi aprimorada utilizando o recurso de Data Annotations, focando na experiência do usuário (UX) e na integridade das informações:

1. Mensagens de Erro (UX): A customização das mensagens foi realizada através da propriedade `ErrorMessage`. Exemplo prático: `[Required(ErrorMessage = "O campo Nome é obrigatório. Por favor, preencha-o.")]`.
2. Opcionalidade de Campos: Para tornar o campo "Complemento" opcional, omitiu-se a anotação `[Required]`. Em C#, utilizando *nullable reference types*, a declaração adotada foi: `public string? Complemento { get; set; }`.
3. Restrição de Comprimento: A limitação de caracteres foi implementada com a anotação `[StringLength]`. Para o CEP, utilizou-se: `[StringLength(9, MinimumLength = 8, ErrorMessage = "O CEP deve ter entre 8 e 9 caracteres.")]`.
4. Validação de Formato (Máscara): Embora a máscara visual seja responsabilidade do front-end (via JavaScript/jQuery), a validação de formato no back-end foi assegurada pela anotação `[RegularExpression]`. Exemplo para o CEP: `[RegularExpression(@"^\d{5}-\d{3}$", ErrorMessage = "Formato de CEP inválido.")]`.

Questões Conceituais (Slide 28):

1. Sobrecarga (Overloading): Princípio da Programação Orientada a Objetos que faculta a coexistência de múltiplos métodos homônimos em uma mesma classe, condicionado à distinção de suas assinaturas (quantidade, tipo ou ordem dos parâmetros).

2. Aplicação Prática da Sobrecarga: Este conceito foi materializado no ``AlunoController``, especificamente no método ``Edit``. Implementou-se ``Edit(int id)`` para requisições GET e ``Edit(int id, Aluno aluno)`` para o processamento de requisições POST.
3. Justificativa para Lista Estática: A utilização do modificador ``static`` foi imperativa para garantir a persistência dos dados em memória entre requisições HTTP distintas. Dada a natureza ``stateless`` do protocolo HTTP e a instanciação do Controller a cada requisição, a ausência deste modificador resultaria na reinicialização da lista e consequente perda dos registros.
4. Acrônimo CRUD: Refere-se às operações basilares de persistência: ``Create`` (Criar), ``Read`` (Ler/Consultar), ``Update`` (Atualizar) e ``Delete`` (Excluir).
5. Resolução de Rotas pelo Framework: O ASP.NET Core distingue métodos homônimos baseando-se nos verbos HTTP e atributos de roteamento. O método de exibição responde ao verbo GET por padrão, enquanto o método de processamento é explicitamente decorado com ``[HttpPost]``.
6. Nomenclatura do Método Delete: O método ``Delete`` (GET) e o método de exclusão efetiva (POST) demandam o mesmo parâmetro (``int id``). Como a linguagem C# não permite sobrecarga com assinaturas idênticas, o método POST foi renomeado para ``DeleteConfirmed(int id)`` e decorado com ``[ActionName("Delete")]`, instruindo o framework a mapeá-lo corretamente para a ``action`` "Delete".

Atividades Práticas (Slides 20 e 23): Operações de Detalhe e Exclusão

Foram implementados os métodos ``Details(int id)``, ``Delete(int id)`` (GET) e ``DeleteConfirmed(int id)`` (POST) no ``AlunoController``. As Views correspondentes foram geradas com o auxílio do assistente do Visual Studio, permitindo a visualização ``read-only`` dos dados e a confirmação prévia à exclusão.

2.4. Arquitetura e Desacoplamento (Arquivo 4)

Atividades Práticas (Slides 3 a 12): Padrão Repository e Injeção de Dependência

A etapa final do desenvolvimento focou na refatoração arquitetural, visando o desacoplamento do código através do padrão Repository e da Injeção de Dependência (DI).

- Contrato (Interface): Definiu-se a interface ``IALunoRepository``, estabelecendo as assinaturas dos métodos assíncronos necessários (``GetAll``, ``GetById``, ``Create``, ``Edit``, ``Delete``, ``Exists``).
- Implementação Concreta: A classe ``InMemoryAlunoRepository`` foi criada para implementar a interface, gerenciando a coleção estática de alunos de maneira ``thread-safe`` mediante a utilização de blocos ``lock``.
- Injeção de Dependência: O ``AlunoController`` foi reestruturado para receber a dependência ``IALunoRepository`` via construtor. Consequentemente, as operações CRUD

foram atualizadas para o paradigma assíncrono (`async Task<IActionResult>`), delegando o acesso a dados ao repositório.

- Configuração de Serviços: O repositório foi devidamente registrado no contêiner de DI da aplicação (arquivo `Program.cs`), adotando o ciclo de vida Singleton:
`builder.Services.AddSingleton<IAlunoRepository, InMemoryAlunoRepository>()`.

3. Repositório de Código-Fonte

A integridade do código-fonte, contemplando todas as implementações e refatorações descritas neste relatório técnico, encontra-se preservada e acessível no repositório GitHub do discente.

Acesso ao Repositório: <https://github.com/LOLOXI7/AVD1-PROGRAMA-O-ORIENTADA-A-OBJETOS-II>

4. Considerações Finais

A execução das atividades propostas no módulo AVD1 proporcionou uma consolidação substancial, tanto teórica quanto prática, dos alicerces do framework ASP.NET Core MVC. A progressão metodológica do projeto — partindo da concepção de formulários básicos até a estruturação de um CRUD completo, enriquecido com validações rigorosas (Data Annotations) e padrões arquiteturais consolidados (Repository e Injeção de Dependência) — evidenciou a capacidade e a robustez da tecnologia empregada.

Conclui-se que a adoção de práticas como a separação de responsabilidades e o desacoplamento de componentes resulta em uma aplicação de alta manutenibilidade. A arquitetura estabelecida encontra-se, portanto, adequadamente preparada para futuras evoluções, notadamente a integração com sistemas de gerenciamento de banco de dados relacionais através de ORMs, como o Entity Framework.

5. Referências Bibliográficas

1. MICROSOFT. Introdução ao ASP.NET Core MVC. Disponível em:
<https://learn.microsoft.com/pt-br/aspnet/core/tutorials/first-mvc-app/start-mvc>. Acesso em: fevereiro de 2025.
2. MICROSOFT. Validação com validadores de dados. Disponível em:
<https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions-1/models-data/validation-with-the-data-annotation-validators-cs>. Acesso em: fevereiro de 2025.

3. BYTEHIDE. Data Annotation in C#: your complete guide. Disponível em: [\[https://www.bytehide.com/blog/data-annotations-in-csharp\]](https://www.bytehide.com/blog/data-annotations-in-csharp)(<https://www.bytehide.com/blog/data-annotations-in-csharp>). Acesso em: fevereiro de 2025.
4. DIO. O que é CRUD?. Disponível em: [\[https://www.dio.me/articles/o-que-e-crud\]](https://www.dio.me/articles/o-que-e-crud)(<https://www.dio.me/articles/o-que-e-crud>). Acesso em: fevereiro de 2025.
5. MICROSOFT. Injeção de dependência no ASP.NET Core. Disponível em: [\[https://learn.microsoft.com/pt-pt/aspnet/core/fundamentals/dependency-injection\]](https://learn.microsoft.com/pt-pt/aspnet/core/fundamentals/dependency-injection)(<https://learn.microsoft.com/pt-pt/aspnet/core/fundamentals/dependency-injection>). Acesso em: março de 2026.
6. MICROSOFT. Implementando o repositório e a unidade de padrões de trabalho em um aplicativo MVC ASP.NET. Disponível em: [\[https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application\]](https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application)(<https://learn.microsoft.com/pt-br/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application>). Acesso em: março de 2026.